

INTER-IIT TECH MEET 15.0

Selection Bootcamp – NLP

Submission

A single file `<rollno>_nlp_bootcamp.zip` containing:

<code>code/</code>	notebook(s) or scripts; <code>README.md</code> with the exact commands to reproduce each task
<code>predictions/</code>	files listed under each problem
<code>results.json</code>	filled-in copy of the template: seed, settings, method names, validation numbers
<code>report.md</code>	answers to the fixed questions (Q1–Q5), ≤ 100 words each

Problem Statement – Finding Evidence for Scientific Claims (SciFact)

Problem Statement

Given a scientific claim (e.g. “*Drug X reduces the risk of Y*”), find the abstracts in a corpus that support or contradict it, then decide which. This is the retrieval core of every RAG system. Lexical and dense retrieval fail in different ways:

- ▶ **Lexical search (BM25)** is a strong baseline but misses paraphrases.
- ▶ **Dense embeddings** capture paraphrase but can miss rare technical terms.

Your job is to build retrievers, compare them, improve them, and then verify claims against the retrieved evidence.

Rules

- ▶ **Models:** open-source **encoder** models only (bi-encoders, cross-encoders, NLI classifiers), with **at most 400M parameters each**. Pretrained weights are allowed. No generative LLMs.
- ▶ **Tuning and selection:** use only the training claims and the provided validation split. Your “best” system at any stage is the one with the highest **validation nDCG@10**.

Tasks

Task 1 – Lexical Baseline

- ▶ Explore the corpus and the claims.
- ▶ Implement BM25 (a library is allowed). Tune it on the validation split.

Task 2 – Dense Retrieval

- ▶ Retrieve with **two** pretrained embedding models: one general-purpose and one scientific or biomedical.
- ▶ Record index time, query latency and memory use in `results.json`.

Task 3 – Combine & Rerank

- ▶ Build a hybrid of BM25 and dense retrieval (e.g. rank fusion or score interpolation).
- ▶ Rerank your best system's top- k results with a cross-encoder.

Task 4 – Domain Fine-Tuning

- ▶ Fine-tune a bi-encoder on the training claims twice: once with random negatives, once with hard negatives mined from your own Task 1–2 systems.

Task 5 – Claim Verification & Error Analysis

- ▶ Using your best retriever, predict for each claim which of its top abstracts SUPPORT or CONTRADICT it. Predicting none means NOT ENOUGH INFO.
- ▶ Use an NLI-style classifier, either zero-shot or fine-tuned on the training claims.
- ▶ Submit predictions for both the **evaluation** claims and the **validation** claims.
- ▶ On the **validation** claims, tag every claim whose predicted (abstract, label) pairs do not exactly match the gold pairs with **one** error category.

Expected Outcomes

- ▶ Seven retrieval runs, which together form an automatic ablation table (BM25 → dense → hybrid → rerank → fine-tuned).
- ▶ A claim-verification stage evaluated end-to-end.
- ▶ Categorized errors showing where the pipeline fails.

Dataset

Source: SciFact (Wadden et al., EMNLP 2020; AllenAI), with labeled claims and a corpus of 5,183 abstracts.

What's provided (in `data/`)

- ▶ `corpus.jsonl`: 5,183 abstracts (`doc_id`, `title`, `abstract` as a list of sentences)
- ▶ `train_claims.jsonl`: 647 claims with gold evidence (`doc_id`, `label`, `sentences`)
- ▶ `val_claims.jsonl`: 162 claims with gold evidence, for tuning and model selection
- ▶ `eval_claims.jsonl`: 300 claims, text only. No labels are provided.
- ▶ `results_template.json`: copy it to `results.json` and fill it in
- ▶ `check_format.py`: validates your submission's file names and formats

Note

A claim with an empty `evidence` list is NOT ENOUGH INFO (NEI). Retrieval metrics on the eval claims are computed only over claims with at least one gold abstract. You are not told which ones those are, so your files must still cover **all 300 eval claims**.

Evaluation

Retrieval run files (in `predictions/`)

- ▶ **Files:** `t1_bm25.trec`, `t2_general.trec`, `t2_scientific.trec`, `t3_hybrid.trec`, `t3_rerank.trec`, `t4_random.trec`, `t4_hard.trec` (eval claims), plus `val_bm25.trec` and `val_general.trec` (validation claims, for Q1).
- ▶ **Format:** whitespace-separated lines, `claim_id Q0 doc_id rank score run_name`.
- ▶ Each file must contain **all** claims of its split (300 eval claims, or 162 validation claims), with **exactly 100 unique doc_ids per claim**, ranks 1 to 100.
- ▶ Scores must not increase as rank increases. Ties are fine, but ranks must still be unique.
- ▶ **Reranking (`t3_rerank.trec`):** rerank the top- k of your best first-stage system with the cross-encoder, and place those k documents at ranks 1 to k in reranker order. Fill ranks $k+1$ to 100 with the remaining documents from the first-stage ranking, in their original order and without duplicates. Report k in `results.json`.
- ▶ Claim IDs and doc IDs are integers.

Verification and error files

- ▶ `t5_verification.csv` (evaluation claims) and `t5_val_verification.csv` (validation claims).
 - Format: `claim_id,doc_id,label`, at most 3 rows per claim.
 - `label` is SUPPORT or CONTRADICT. A claim with no rows means NOT ENOUGH INFO.
- ▶ `t5_val_errors.csv`, format `claim_id,category`.
 - Allowed categories: `retrieval_miss`, `negation`, `numerical`, `entity_mismatch`, `needs_multiple_docs`, `other`.

Scoring

File	Scored by
Seven <code>.trec</code> run files	nDCG@10 (primary), Recall@100, MRR
<code>t5_verification.csv</code>	Precision / recall / F1 over (claim, doc, label) triples
<code>t5_val_verification.csv</code> + <code>t5_val_errors.csv</code>	Checks that every tagged claim is an actual validation error; category counts

Report Questions

(`report.md`, ≤ 100 words each; all analysis on **validation** claims)

- Q1.** Give two claim IDs where BM25 has higher per-claim nDCG@10 than `t2_general`, and two where `t2_general` beats BM25. Explain why. *To make this checkable, also include `val_bm25.trec` and `val_general.trec` in predictions/.*
- Q2.** Which Task 3 component gave the largest validation gain? What did it cost in latency?
- Q3.** Did hard negatives help? Did you see overfitting, and how did you detect it?
- Q4.** What fraction of your validation verification errors are `retrieval_miss`?
- Q5.** What is the single change you would try next, and why?